Is Narration Studio harness-engineered?

Narration Studio as a harness-centric MAF application: explicit production harness, tools as the only I/O surface, MAF agents as optional drivers.

> **Confirmed** means the behavior is in this repository's source. **Inferred** means it comes from Microsoft Agent Framework (MAF) or from the industry definition of an agent harness.

Verdict

Yes, for the video factory. The outer production harness is explicit: `ProductionHarness` owns the run, `ProductionTools` is the only I/O surface, `HarnessChecks` can fail the process if artifacts are wrong, and `events.jsonl` records tool calls. MAF handoff/sequential remain optional drivers. Default CLI is still `handoff`. Direct (`DirectExecutor`) is the reference executor and the failover path.

The coding-agent outer harness is present in thin form: root `AGENTS.md` , `.cursor/rules/studio-harness.mdc` , and `tests/NarrationStudio.Tests` .

What "harness" means here

An agent harness is the scaffold around a model: the loop, tools, context, and controls. MAF is **Harness-as-a-Service** (**inferred**): this app customizes it rather than writing its own chat-tool loop.

Piece	In this app now
Loop	`ProductionHarness` → MAF `WorkflowHost` or `DirectExecutor`
Tools	`ProductionTools` only; events via `HarnessEvents`
Context	`ProductionContextProvider` + timestamped snapshots; option-driven names
Control	Mutex, tool throws, MAF chat guardrails, failover to direct, `HarnessChecks` , cancellation via `ApplicationStopping`

Production harness (current)

Loop

`ProductionApp` is a thin CLI (mutex, paths, `--orchestration` , banner including `Harness : MAF+tools`). `ProductionHarness.RunAsync` copies stub narration if configured, then runs MAF or `DirectExecutor` , failsover if the merged file is missing, then `HarnessChecks.Verify()` .

Default CLI mode is `handoff` . `--orchestration direct` skips MAF.

Tools

`ProductionTools` writes every PNG, MP3, and MP4. Agents get `AIFunction` wrappers of the same methods. New stages must be new tool methods.

Each public tool call appends a JSONL line to `output/agent-file-memory/production-notes-{RunId}/events.jsonl` (`name` , `elapsedMs` , `ok` , `error`).

Ollama, TTS, and ffmpeg take `ApplicationStopping` . `Studio:UseStubSlidePhrases` skips per-slide Ollama (independent of `UseStubNarration`).

Context

Specialists still get `ProductionContextProvider` plus MAF `FileMemoryProvider` (agent modes only). Caps and file-name ranges interpolate `StudioOptions` (`StudioNaming`). See `docs/memory-vs-mem0.md` .

Control

Control	Where
Mutex	`ProductionApp`
Chat blocked terms / PII redact	`ProductionGuardrails` (MAF only)
Script/slide caps	`ProductionTools` throws / `Take()` — all modes
Failover to direct	`ProductionHarness`
Post-run artifact check	`HarnessChecks` — fails the process if not `complete` with files on disk
Wipe media on start	`PipelineStore.Initialize`

The unused slide-count “trim” log was removed. Chat guardrails still do not wrap `DirectExecutor` .

Coding-agent outer harness

- `AGENTS.md` — cwd, commands, tools own I/O, do not hardcode 8
- `.cursor/rules/studio-harness.mdc`
- `dotnet test` — `tests/NarrationStudio.Tests`

How modes use the harness

```
ProductionApp (CLI + banner)
    → ProductionHarness
        → (optional stub narration copy)
        → handoff/sequential: WorkflowHost
              → if no merged MP4 → DirectExecutor
        → or direct / TryCreate fail → DirectExecutor
        → HarnessChecks.Verify  (throw if artifacts wrong)
```

The durable contract is `ProductionTools` + `DirectExecutor` + `HarnessChecks`. MAF is the inner runtime.

Remaining (not this refactor)

Resume-without-wipe, critic agent, `--approve` before TTS, mem0, parallel TTS, swapping MAF. See `docs/agentic-system.md`.

Key files

File	Role
`src/NarrationStudio/Harness/ProductionHarness.cs`	Run policy, failover, checks
`src/NarrationStudio/Harness/HarnessChecks.cs`	Artifact success predicate
`src/NarrationStudio/Harness/HarnessEvents.cs`	`events.jsonl`
`src/NarrationStudio/ProductionApp.cs`	CLI, mutex, banner
`src/NarrationStudio/Orchestration/DirectExecutor.cs`	No-agent tool loop
`src/NarrationStudio/Tools/ProductionTools.cs`	Shared I/O; cancellation; events
`src/NarrationStudio/Agents/AgentFactory.cs`	Option-driven crew prompts
`src/NarrationStudio/Agents/WorkflowHost.cs`	Handoff / sequential
`AGENTS.md`	Coding-agent outer harness